第 11 章：虚拟环境、pip 与 uv——让 Python 项目真正可重建

一句话先说透：**Python 启动时会先选定一个具体解释器，`import` 只能看到这个解释器环境中的包。**`requirements.txt`、`pyproject.toml` 和 `uv.lock` 负责描述“应该安装什么”，但它们不会在 `python main.py` 启动时自动下载依赖。

这和 Java 项目最不一样的地方是：Maven/Gradle 通常会在构建或运行任务前解析依赖并组装 classpath；普通 Python 解释器只负责执行代码，不会先读取项目声明并补齐环境。

学完本章，你应该能回答四个问题：

1. 当前究竟是哪个 Python 在运行？
2. 一个第三方库安装到了哪个环境？
3. 项目声明、锁文件和 `.venv` 各自保存什么？
4. 为什么“明明安装过”仍然会出现 `ModuleNotFoundError`？

11.1 先建立 Java → Python 心智模型

下面的类比用于快速定位职责，但都不是严格的一一对应。

Java 世界	Python 世界	相似点与关键边界
选定的 JDK / `java` 可执行文件	Python 解释器，如 `.venv/bin/python`	都决定实际运行时；Python 解释器还直接决定默认第三方包目录
运行时 classpath	`sys.path`	都决定代码查找范围；Python 在运行时按顺序查找并执行模块
Maven/Gradle 依赖声明	`requirements.txt` 或 `pyproject.toml`	都表达项目需要什么；普通 `python` 不会自动读取并安装它们
Gradle dependency locking 等锁定结果	`uv.lock`	都保存一次解析后的精确依赖图；锁文件本身不包含已安装文件
Maven 坐标中的 artifact	Python 发行包（distribution）	都是安装工具处理的发布单元；

README README README
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html
目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代

Maven 坐标中的 artifact	Python 发行包（distribution）	都是安装工具处理的发布单元； 发行包名不一定等于代码导入名
项目私有运行目录	`.venv` 虚拟环境	都承载可丢弃的本机状态；`.venv` 不是容器， 也不是可移植产物

一次完整运行可以理解成下面这条链路：

```
选择 Python 解释器
    → 创建或找到虚拟环境
    → 根据依赖声明解析版本
    → 把发行包安装进这个环境
    → 用同一个环境的解释器启动程序
    → import 按 sys.path 查找模块
```

排错时不要把整条链路都归结为“pip 有问题”，而要先判断错误发生在哪一步。

11.2 六个名字相近、职责不同的对象

解释器：真正执行代码的程序

`python` 只是 shell 根据 `PATH` 找到的命令名。机器上可以同时有系统 Python、仓库根 `.venv` 的 Python、某个子项目 `.venv` 的 Python，以及 IDE 单独选中的 Python。

先用运行时证据确认它，而不是看终端提示符猜：

```
python -V
python -c 'import sys; print(sys.executable); print(sys.prefix); print(sys.base_prefix)'
```

- `sys.executable`：当前进程由哪个解释器入口启动。
- `sys.prefix`：当前环境的根目录。
- `sys.base_prefix`：创建虚拟环境所依赖的基础 Python 目录。
- `sys.prefix != sys.base_prefix`：通常表示当前进程正在虚拟环境中。

这类似于打印 Java 进程实际使用的 `java.home`，比“我记得 IDE 选过某个 SDK”可靠。

虚拟环境：项目专属的第三方包空间

`venv` 会基于一个已有 Python 创建环境入口、`pyvenv.cfg` 和独立的 `site-packages`。它通常

虚拟环境：项目专属的第三方包空间

`venv` 会基于一个已有 Python 创建环境入口、`pyvenv.cfg` 和独立的 `site-packages`。它通常复用基础解释器及标准库，不是复制一整套操作系统。

```
# 在一个新项目目录中创建环境；本仓库要求 Python 3.14。
python3.14 -m venv .venv

# macOS / Linux：激活后，python 会优先指向 .venv。
source .venv/bin/activate
python -c 'import sys; print(sys.executable)'
deactivate

# 不激活也完全可以，直接指定解释器最明确。
.venv/bin/python -c 'import sys; print(sys.executable)'
```

Windows PowerShell 对应的入口通常是 `.venv\Scripts\python.exe`，激活脚本是 `.venv\Scripts\Activate.ps1`。

激活的本质只是修改**当前 shell** 的 `PATH`，并设置 `VIRTUAL_ENV` 等辅助信息。它不会修改系统 Python，不会改变已经启动的 IDE 或 Python 进程，也不会自动安装项目依赖。虚拟环境还不会隔离环境变量、网络、业务文件或操作系统进程，所以它不能与 Docker 容器等同。

直接运行 `.venv/bin/python` 时，即使 `VIRTUAL_ENV` 没有设置，虚拟环境仍然有效。

`.venv` 应被视为可重建的本机产物。里面的启动脚本可能记录绝对路径，二进制扩展还依赖 Python 版本、操作系统和 CPU，所以不要复制 `.venv` 或 `site-packages` 给同事。应该提交源码、Python 版本要求和依赖声明，让对方重新创建环境。

发行包与导入包：安装名不等于代码中的名字

本章安装的发行包是：

```
python-dotenv==1.2.3
```

代码使用的导入包却是：

```
from dotenv import dotenv_values
```

`python-dotenv` 是 pip/uv 和包索引认识的发行包名，`dotenv` 是 Python 运行时认识的导入名。二者没有机械转换规则。连字符不能出现在 Python 标识符中，而擅自改成 `python_dotenv` 仍然是错的；一个发行包也可能提供多个导入包。

这可以类比为“artifactId 不等于 Java package”，但 Python 没有要求二者遵守同一套命名规则。应以库的官方用法或安装元数据为准。

包索引：只参与安装，不参与 import

PyPI 是常见的公共包索引，也可以配置公司私服。pip/uv 在安装阶段查询索引、选择兼容版本并下载分发文件。`import` 阶段不会访问索引，更不会发现缺包后自动联网安装。

`sys.path` 与 `site-packages`：查找范围和安装位置

第三方包通常安装进环境的 `site-packages`，而 `sys.path` 是当前进程实际使用的模块搜索路径列表。前者是常见安装位置，后者才决定本次 `import` 会到哪里找。

```
import sys

print(sys.executable)
for position, path in enumerate(sys.path):
    print(position, path)
```

脚本目录、当前工作目录、`-I` / `-P` 参数和环境配置都可能影响搜索路径。因此“文件确实在磁盘上”不代表当前进程一定能导入它。

11.3 黄金规则：始终让 pip 绑定到目标解释器

推荐这样安装：

```
python -m pip install "python-dotenv==1.2.3"
```

`python -m pip` 的意思是“让刚才选中的这个 Python 执行它能看到的 pip 模块”。如果只写 `pip install ...`，shell 会再次根据 `PATH` 独立寻找 `pip`，它可能属于另一套 Python。

用下面三条命令可以把解释器、pip 和安装目录对齐：

```
python -c 'import sys; print(sys.executable)'
```

用下面三条命令可以把解释器、pip 和安装目录对齐：

```
python -c 'import sys; print(sys.executable)'
python -m pip --version
python -m pip show python-dotenv
```

`pip --version` 的输出同时包含 pip 所在路径和对应的 Python 版本。两个解释器都显示“Python 3.14.7”，也不代表它们共享已安装包；必须继续比较路径。

命令	它实际证明什么	它不能证明什么
`python -m pip install X`	把 X 及所需依赖安装到目标环境	不会自动把 X 写回项目声明
`python -m pip install -r requirements.txt`	满足清单中的安装要求	通常不会删除清单之外的额外包
`python -m pip show X`	当前环境存在发行包 X 的元数据	不代表导入名也叫 X
`python -m pip list`	当前环境安装了哪些发行包	不区分直接依赖和间接依赖的业务意图
`python -m pip check`	已安装包声明的依赖是否缺失或冲突	不会运行应用， 也检查不了本地文件遮蔽

11.4 `requirements.txt`：安装清单，不是自动执行的构建文件

本章的 `demo_project/requirements.txt` 只有一个直接运行依赖：

```
python-dotenv==1.2.3
```

`==1.2.3` 固定这个直接依赖的版本。若写成 `>=1.2.3,<2`，以后重新解析时可以选择范围内的其他版本。依赖所需的其他包叫间接依赖，pip 会根据发行包元数据继续解析。

`requirements.txt` 可以先类比 Maven/Gradle 的依赖清单，但边界很重要：pip 读取它完成安装，普通 `python main.py` 不会读取它，也不会自动把当前环境同步成“只包含清单中的内容”。

`pip freeze` 为什么不能替代依赖设计

```
python -m pip freeze > environment-snapshot.txt
```

2502x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html.

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商

`freeze` 记录当前环境的安装快照。它可能包含临时工具、间接依赖、本地路径，以及与当前机器有关的信息，却不说明“项目为什么需要这个包”。它更像对当前 classpath 结果做快照，而不是重新编写一份有业务意图的构建文件。

快照适合排查和临时复现；正式项目仍应维护清晰的直接依赖，并通过干净环境验证可重建性。

11.5 `pyproject.toml` 表达意图，`uv.lock` 保存解析结果

本章演示项目的核心声明是：

```
[project]
name = "chapter11-config-demo"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = ["python-dotenv>=1.2.3,<2"]

[dependency-groups]
dev = []

[tool.uv]
package = false
```

- `name` 和 `version` 是项目发行元信息，不保证存在同名导入包。
- `requires-python = ">=3.14"` 声明兼容范围，不会把当前 `python` 自动切换成 3.14。
- `dependencies` 是应用直接运行依赖，类似 Maven 的普通运行依赖。
- `dependency-groups.dev` 用于测试、格式化等开发依赖；本章暂时为空。
- `package = false` 告诉 uv：这是直接运行脚本的应用，不必把应用自身构建安装成发行包。

`pyproject.toml` 表达“允许并需要什么”，而 `demo_project/uv.lock` 保存 uv 已解析出的精确版本、来源和文件哈希。本章锁文件最终选择 `python-dotenv 1.2.3`。

状态	本章对应物	应否提交 Git
依赖意图	`pyproject.toml`	应提交
精确解析结果	`uv.lock`	本章这种应用项目应提交
解释器选择约定	`.python-version` 中的 `3.14`	可以提交
本机实际安装结果	`.venv/`	不提交，按声明与锁重建

wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html

相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

这类似"构建声明 → 依赖锁定 → 本机产物"三层。`uv.lock` 不是 `.venv` 的压缩包，也不是 `pip install -r` 能直接读取的 requirements 文件；通常应由 uv 维护，不要手改哈希和版本。

`.python-version` 会被 uv 等工具用于选择 Python，但普通 Python 不会读取它。它只写了 `3.14`，没有固定补丁版本；项目是否允许某解释器仍以 `requires-python` 等约束为准。

11.6 uv 把声明、锁定和环境同步成一套工作流

uv 可以先类比为"环境管理 + 依赖解析 + 锁定 + 命令运行"的组合工具，但它不是 Maven 的 Python 语法版。它仍然要创建 Python 环境，运行时也仍然遵守 Python 的 import 规则。

下面以本章锁文件使用的 uv 0.12.10 行为为基线：

命令	声明	锁文件	`.venv`
`uv add "python-dotenv>=1.2.3,<2"`	添加或调整直接依赖	更新	同步
`uv remove python-dotenv`	删除直接依赖	更新	同步；若仍是间接依赖则可能保留
`uv add --dev pytest`	写入开发依赖组	更新	默认同步开发依赖
`uv lock`	不改变直接依赖意图	创建或更新解析结果	不安装包
`uv sync`	不改变直接依赖意图	必要时更新	使环境匹配本次依赖选择，默认移除多余包
`uv run python main.py`	通常不改直接依赖	必要时更新	确保环境可运行后启动命令

两个常见误判：`uv lock` 成功不代表 `.venv` 已存在；`uv run` 也不是纯启动器，首次运行可能解析、下载和安装依赖。

用于 CI 或可重复验证时，可以加 `--locked`：

```
uv lock --check
uv sync --locked
uv run --locked python main.py
```

`--locked` 表示锁文件缺失或与声明不一致时失败，不允许命令悄悄改锁；但 `sync` / `run` 仍然可能创建或修改 `.venv`，也可能为了安装锁定依赖而联网。

2502x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家

uv 管理的虚拟环境可以没有 pip，因为 uv 自己能安装和检查包。此时 `python -m pip --version` 报 `No module named pip` 不代表环境坏了，可以使用：

```
uv pip list --python .venv/bin/python
uv pip check --python .venv/bin/python
```

不要在 uv 项目里随手用 pip 安装依赖却不更新声明。那只会改变本机环境： `uv run` 可能暂时还能看到这个额外包，而一次干净重建或精确 `uv sync` 会暴露声明缺失。正式增删依赖应使用 `uv add / uv remove` 。

11.7 本章的小应用：像读取 Java `Properties` 一样读取 `.env`

本章用 `python-dotenv` 读取本地 Agent 配置。这可以先类比 Java 的 `Properties` ：文件中的值读出来仍然是字符串，业务代码负责必填检查和类型转换。但 `.env` 的语法、插值规则和 API 并不等同于 `.properties` 。

下面是一个无需预读项目代码也能理解的最小例子：

```
from io import StringIO

from dotenv import dotenv_values

config = dotenv_values(
    stream=StringIO("AGENT_NAME=文档助手\nTOOL_LIMIT=3\n"),
    interpolate=False,
)

name = config.get("AGENT_NAME")
limit_text = config.get("TOOL_LIMIT")
if not name or limit_text is None:
    raise ValueError("配置缺少必填项")

limit = int(limit_text)
if limit <= 0:
    raise ValueError("TOOL_LIMIT 必须是正整数")

print(name, limit, type(limit).__name__)
```

执行顺序很直接：

2x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

执行顺序很直接：

1. `dotenv_values()` 解析文本并返回类似 `dict` 的映射，不修改 `os.environ`。
2. `get()` 在键不存在时返回 `None`，因此先做必填检查。
3. `TOOL_LIMIT` 原始值是字符串 `"3"`，`int()` 才把它转换成整数。
4. 类型转换成功不代表业务合法，所以还要检查是否为正数。

如果确实希望把配置写入当前进程环境，再考虑 `load_dotenv()`。只需要读取配置时，返回映射的 `dotenv_values()` 副作用更小，也更像显式加载一个 `Properties` 对象。

本章的 `demo_project/main.py` 读取同目录的 `settings.env.example`，它用 `Path(__file__)` 定位配置，所以无论从仓库根目录还是脚本目录启动，都不会把当前工作目录误当成配置目录。示例只展示 `STATUS_URL`，不会发送 HTTP 请求，也不需要 API Key。

11.8 两种完整运行方式：选一套即可

演示项目同时保留 requirements 和 uv 声明，只是为了比较两种工作流。真实项目应选定一份权威依赖声明，避免手工维护两套容易漂移的版本要求。

方式 A：venv + pip

以下命令在**仓库根目录**执行，适用于 macOS / Linux：

```
.venv/bin/python -m venv java_to_python/chapters/11_environments_packages/demo_project/.venv

CH11_PY="java_to_python/chapters/11_environments_packages/demo_project/.venv/bin/python"
"$CH11_PY" -m pip install \
  -r java_to_python/chapters/11_environments_packages/demo_project/requirements.txt
"$CH11_PY" -B java_to_python/chapters/11_environments_packages/demo_project/main.py
```

这里用仓库根 `.venv/bin/python` 创建子项目环境，是为了明确沿用仓库的 Python 3.14。安装和运行又使用同一个 `CH11_PY`，不会出现“装进 A、运行 B”的歧义。

预期核心输出包括：

```
python-dotenv 版本: 1.2.3
助手名称: 文档与记录查询助手
原始 TOOL_LIMIT: '3' str
```

```
python-dotenv 版本：1.2.3
助手名称：文档与记录查询助手
原始 TOOL_LIMIT：'3' str
转换后的工具数量上限：3 int
```

方式 B：uv

先确保终端中已有 uv；没有时可按
uv 官方安装说明安装。本章保存的锁文件
由 uv 0.12.10 生成。

在**仓库根目录**执行：

```
uv lock --check --project java_to_python/chapters/11_environments_packages/demo_project
uv sync --locked --project java_to_python/chapters/11_environments_packages/demo_project
uv run --locked --project java_to_python/chapters/11_environments_packages/demo_project \
  python -B java_to_python/chapters/11_environments_packages/demo_project/main.py
```

`--project` 告诉 uv 到哪里找 `pyproject.toml` 、`uv.lock` 和项目环境；它不会自动把 shell 的
当前工作目录切换到那个目录，所以运行命令仍给出了从仓库根开始的脚本路径。

11.9 先读懂，再用仓库脚本验证

只读诊断：不需要安装 `python-dotenv`

```
# 仓库根目录
.venv/bin/python -B java_to_python/chapters/11_environments_packages/example.py
```

`example.py` 只使用标准库，不创建环境、不安装依赖。运行后重点观察：

- `sys.executable` 、 `sys.prefix` 与 `sys.base_prefix` ；
- 根项目的 `requires-python` 和直接依赖声明；
- 当前环境的 `site-packages` 与 `sys.path` ；
- 发行包元数据和 `dotenv` 模块位置；
- `python -m pip` 实际绑定到哪里。

本次改写时核对的仓库根环境是 Python 3.14.7、pip 26.2.1。换机器后路径和补丁版本可以不同，

README README README README README
2x6dg8fpjj0sff40000gp/T/crossnote-f-caba7cce04762bd8ec3c50b948.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

本次改写时核对的仓库根环境是 Python 3.14.7、pip 26.2.1。换机器后路径和补丁版本可以不同，应该验证的是它们之间的对应关系，而不是照抄输出。

导入故障实验：需要已安装 `python-dotenv`

先完成上面的 pip 或 uv 安装，再从仓库根目录执行：

```
java_to_python/chapters/11_environments_packages/demo_project/.venv/bin/python -B \
  java_to_python/chapters/11_environments_packages/import_lab.py
```

`import_lab.py` 会在系统临时目录中验证三类问题：发行包名不等于导入名、包只安装在另一个环境、本地 `dotenv.py` 遮蔽正确包。实验会创建临时空环境和反例文件，并在结束时自动清理；它不会联网安装软件。

11.10 `ModuleNotFoundError` 的证据式排查顺序

遇到“终端能运行，IDE 不能运行”时，按下面顺序收集证据。

第一步：比较实际解释器

在成功入口和失败入口都打印：

```
import sys

print(sys.executable)
print(sys.prefix)
```

路径不同，就先统一解释器，或者在真正要运行的环境中按项目声明安装依赖。

第二步：分别确认发行包和导入名

```
目标解释器 -m pip show python-dotenv
目标解释器 -c 'import dotenv; print(dotenv.__file__)'
```

第一条检查安装元数据，第二条实际执行 import 并打印模块来源。请把“目标解释器”替换成真实路径，例如 `.venv/bin/python`，不要把这四个汉字直接复制到终端。

第三步：检查是否导入了错误文件

```
from pathlib import Path
import sys
import dotenv

print("工作目录:", Path.cwd())
print("搜索路径:", sys.path)
print("模块来源:", dotenv.__file__)
```

如果项目里有 `dotenv.py` 、 `requests.py` 、 `json.py` 、 `typing.py` 等同名文件，它可能比已安装包更早被找到。此时不是“没找到模块”，而是“找到了错误模块”。正确做法是重命名自己的文件，再启动新进程验证；反复重装第三方包通常无效。

第四步：阅读完整 traceback

顶层包找到了，包内部仍可能因为间接依赖缺失、二进制扩展不兼容或初始化代码异常而失败。不要看到 import 失败就一律归为“没有安装”。完整 traceback 会告诉你失败发生在哪一层。

11.11 VS Code 中为什么尤其容易选错环境

本仓库 `.vscode/settings.json` 中有两个独立运行入口：

- Python 扩展默认指向仓库根 `.venv/bin/python` ； 可以通过 **Python: Select Interpreter** 选择其他环境。
- Code Runner 的 Python 命令被明确写成仓库根 `.venv/bin/python` ， 不会自动跟随 Python 扩展刚刚选择的解释器。

因此，即使 Python 扩展已经选择 `demo_project/.venv` ，点击 Code Runner 的 **Run Code** 仍可能使用根环境，而根环境没有安装 `python-dotenv` 。判断依据永远是程序打印的 `sys.executable` ，不是状态栏看起来像选择了哪个环境。

长期在演示项目中开发时，IDE 应选择：

```
java_to_python/chapters/11_environments_packages/demo_project/.venv/bin/python
```

11.12 Java 工程师最容易带过来的误判

11.12 Java 工程师最容易带过来的误判

误判	为什么错	正确做法
“版本号一样， 就是同一个运行环境”	两个 Python 3.14 可以拥有不同 `site-packages`	比较 `sys.executable` 和 `sys.prefix`
“激活后所有工具都会切换环境”	激活只修改当前 shell，已启动的 IDE 不受影响	在每个运行入口打印解释器路径
“pip install 会更新项目声明”	pip 默认只改变当前环境	先维护 requirements，或使用 `uv add`
“pyproject 写了依赖，python 会自动安装”	普通解释器不会执行依赖同步	显式运行 pip/uv 安装或同步命令
“安装名就是 import 名”	发行包和导入包使用不同命名空间	查官方示例，并打印模块 `__file__`
“pip freeze 就是设计好的锁文件”	它只是当前环境快照， 缺少直接依赖意图	维护声明，并在干净环境重建验证
“复制 `.venv` 最省事”	路径、 解释器和二进制扩展可能与机器绑定	复制声明和锁，在目标机器重建
“pip check 成功就说明应用没问题”	它只检查安装元数据中的依赖关系	继续实际 import、运行和测试
“相对路径总是相对源码文件”	`Path("x")` 默认相对当前工作目录	用 `Path(__file__)` 显式定位资源

11.13 版本管理与团队约定

本章的 `.gitignore` 已忽略 `.venv` 、字节码、工具缓存和真实 `.env` 文件。

建议提交源码、项目选定的依赖声明、应用项目的 `uv.lock` 、 `.python-version` ，以及不含密钥的示例配置。不要提交 `.venv/` 、 `__pycache__/` 、下载缓存、真实 `.env` 、API Key 或机器专属配置。

`.gitignore` 只影响尚未跟踪的文件。若敏感文件已经进入 Git 历史，新增忽略规则并不会自动移除它，也不能撤销泄露；应立即轮换密钥并按团队流程处理历史记录。

“安装名就是 import 名”	发行包和导入包使用不同命名空间	查官方示例，并打印模块 `__file__`
“pip freeze 就是设计好的锁文件”	它只是当前环境快照， 缺少直接依赖意图	维护声明，并在干净环境重建验证
“复制 `.venv` 最省事”	路径、 解释器和二进制扩展可能与机器绑定	复制声明和锁，在目标机器重建
“pip check 成功就说明应用没问题”	它只检查安装元数据中的依赖关系	继续实际 import、运行和测试
“相对路径总是相对源码文件”	`Path("x")` 默认相对当前工作目录	用 `Path(__file__)` 显式定位资源

11.13 版本管理与团队约定

本章的 `.gitignore` 已忽略 `.venv` 、字节码、工具缓存和真实 `.env` 文件。

建议提交源码、项目选定的依赖声明、应用项目的 `uv.lock` 、 `.python-version` ，以及不含密钥的示例配置。不要提交 `.venv/` 、 `__pycache__/` 、下载缓存、真实 `.env` 、API Key 或机器专属配置。

`.gitignore` 只影响尚未跟踪的文件。若敏感文件已经进入 Git 历史，新增忽略规则并不会自动移除它，也不能撤销泄露；应立即轮换密钥并按团队流程处理历史记录。

11.14 本章小结

记住下面四句话，就抓住了 Python 环境管理的主线：

1. **解释器决定环境。** 先看 `sys.executable` ，再谈包是否安装。
2. **安装名不等于导入名。** `python-dotenv` 安装后使用 `dotenv` 。
3. **声明、锁和安装结果是三层状态。** `pyproject.toml` 表达意图， `uv.lock` 固定解析，`.venv` 承载本机实际文件。
4. **环境应该重建，不应该复制。** 用同一解释器完成安装和运行，并在干净环境中验证。

掌握这套思路后，后续安装 requests、Pydantic、HTTPX、FastAPI 或 pytest 时只是依赖名称发生变化，诊断方法仍然相同。


example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
demo_project
.python-version
main.py
pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
~/vscode/python-quickstart-for-java/java_to_python/chapters/11_envin
12_http_json
13_typing
14_data_models
15_protocols_generics
example.py
大纲
时间线
Java Projects
学习
1.
2.
3.
4.
11.1
下面的类
选定的
运行时
Maven/
Maven 坐
项目私有
一次完整运行
选择 Python

```
1  3.14
2
```

```python
"""安装 python-dotenv 后，离线读取与脚本放在一起的演示配置。"""

import sys
from importlib import metadata
from pathlib import Path

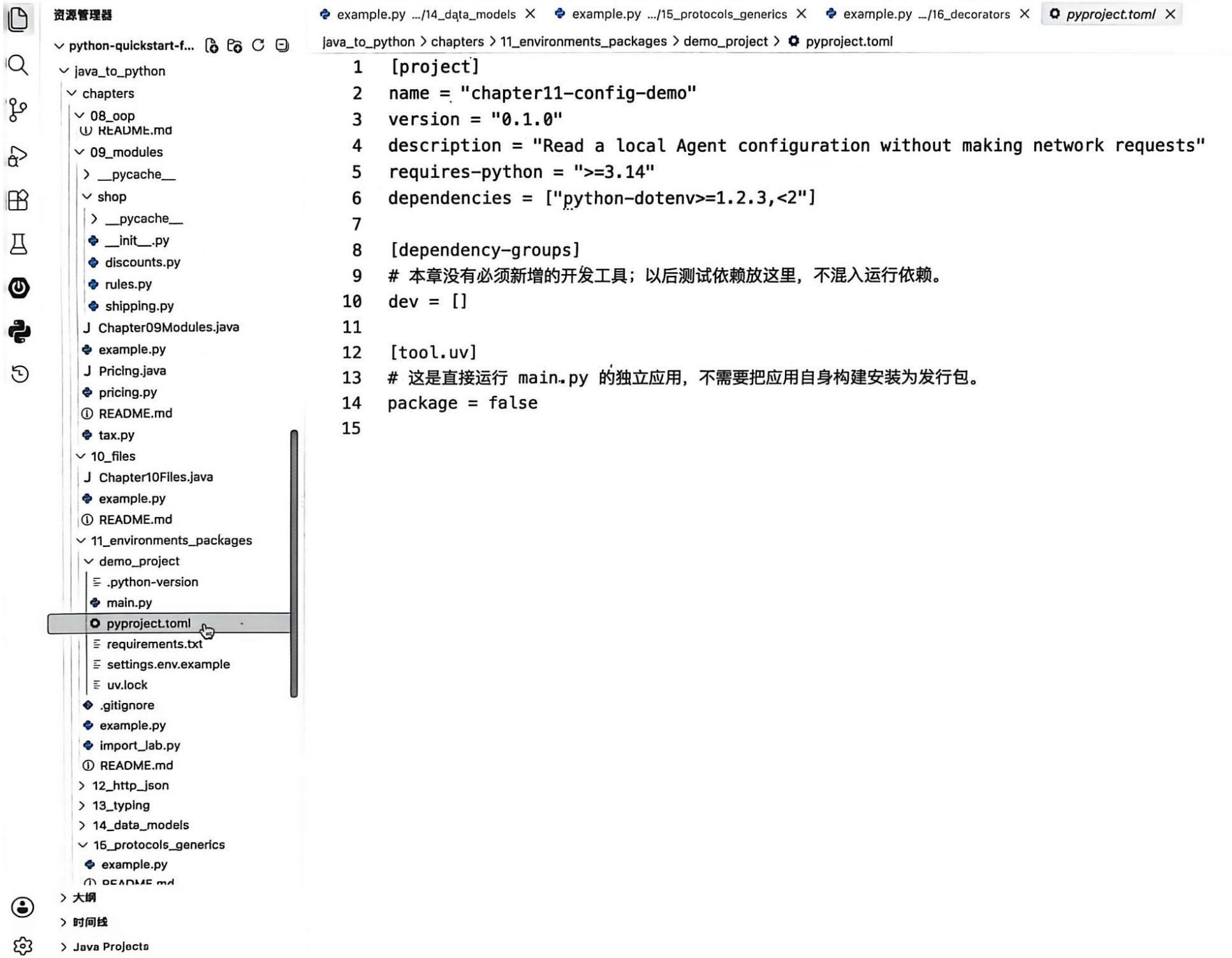
from dotenv import dotenv_values

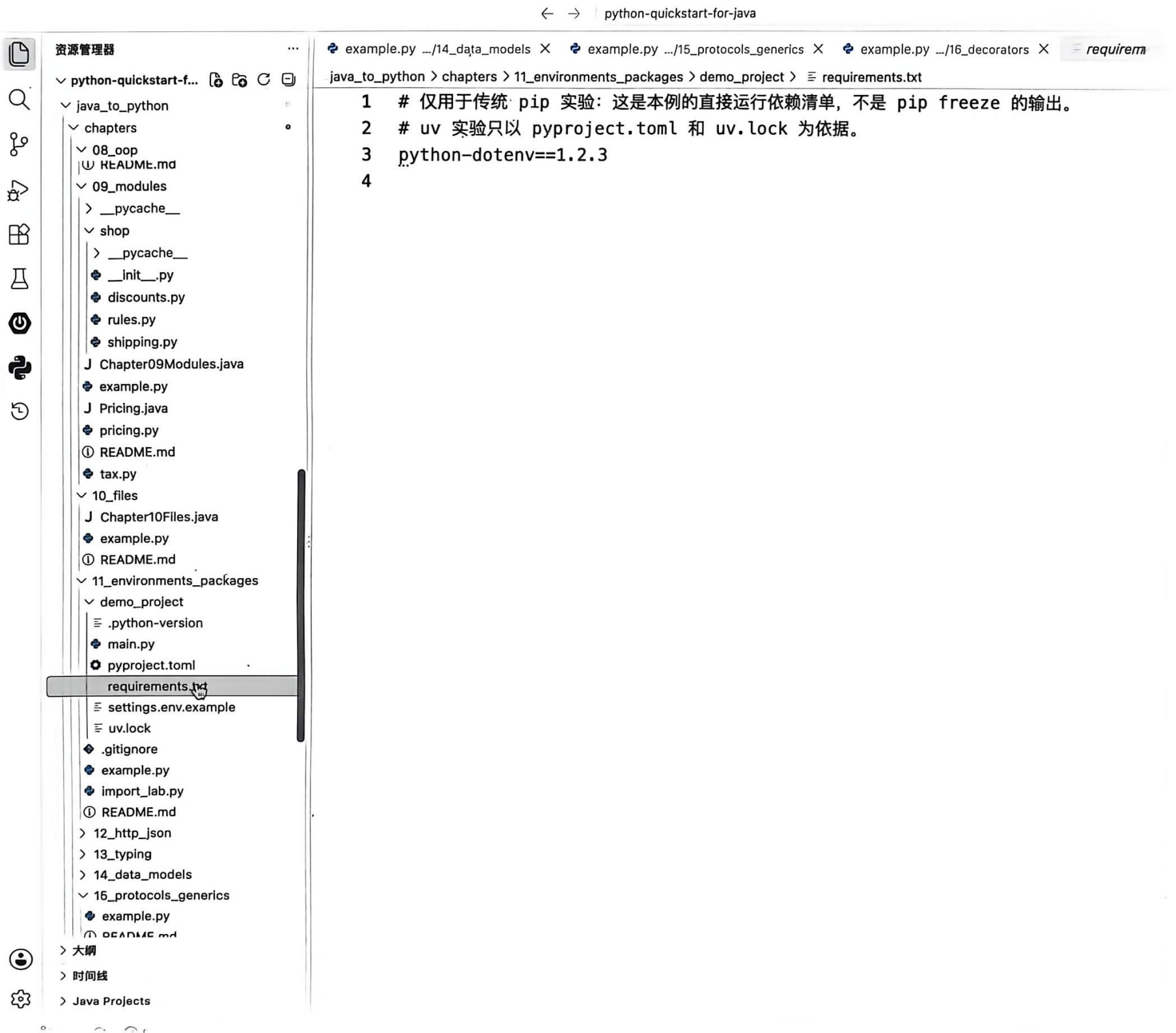

def main() -> None:
    print("\n=== 1. 运行解释器与实际依赖 ===")
    print("sys.executable:", sys.executable)
    print("python-dotenv 版本:", metadata.version("python-dotenv"))
    print("导入名: dotenv")

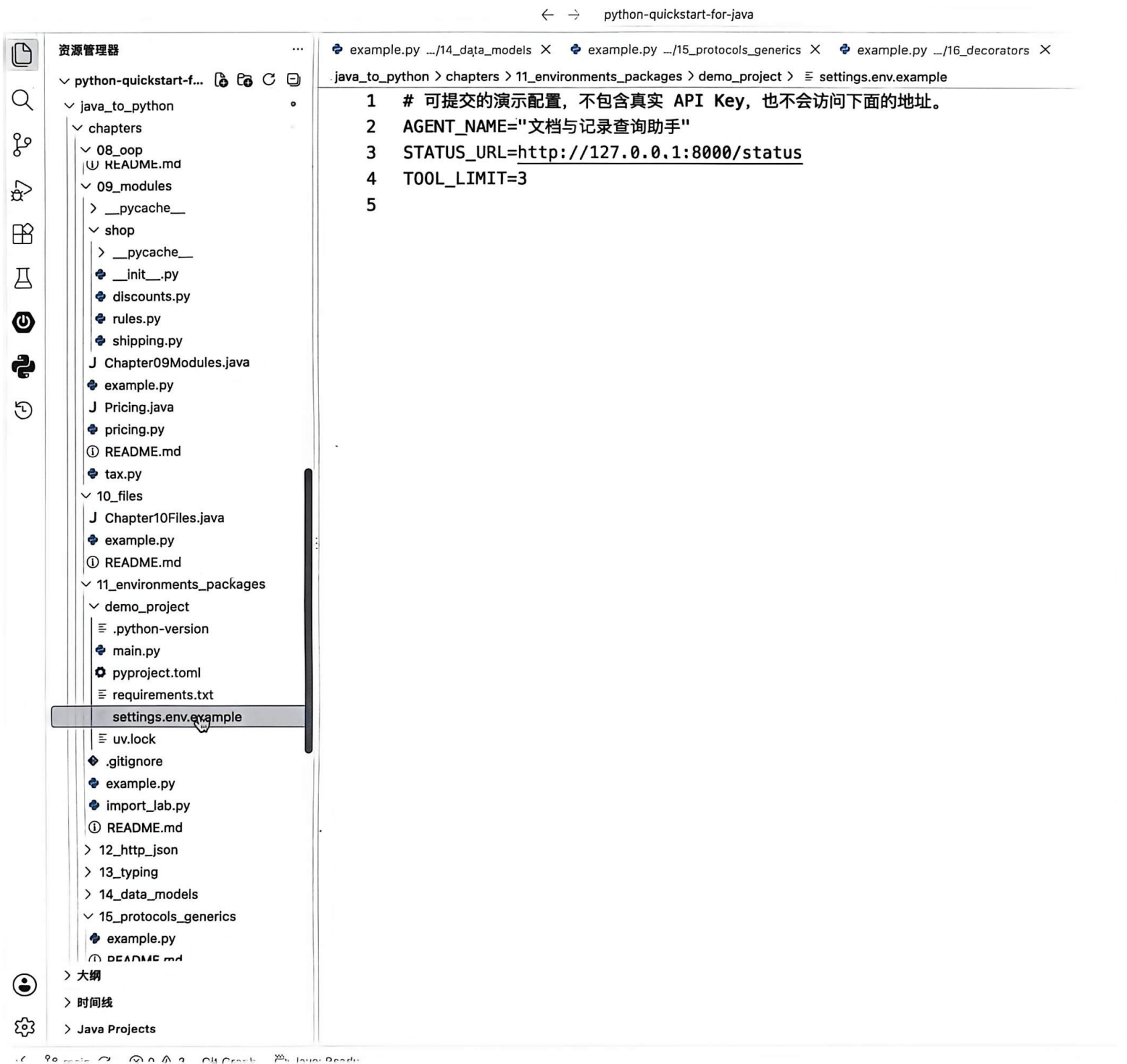
    print("\n=== 2. 读取本地 Agent 配置 ===")
    # 使用脚本位置定位配置：从仓库根目录或其他工作目录运行都读同一份文件。
    config_path: Path = Path(__file__).resolve().with_name("settings.env.example")
    if not config_path.is_file():
        raise FileNotFoundError(f"演示配置缺失：{config_path}")
    # 返回字典即可满足本例；无需 load_dotenv() 修改当前进程的环境变量。
    # 禁用变量插值，让输出只由这份示例文件决定，不读取外部变量来替换配置值。
    config: Any = dotenv_values(config_path, encoding="utf-8", interpolate=False)
    agent_name: Any = config.get("AGENT_NAME")
    status_url: Any = config.get("STATUS_URL")
    limit_text: Any = config.get("TOOL_LIMIT")
    if not agent_name or not status_url or not limit_text:
        raise ValueError("AGENT_NAME、STATUS_URL 和 TOOL_LIMIT 必须有值")

    # 配置解析得到字符串，不会自动把 '3' 变成整数，更不会校验业务上的正数范围。
    tool_limit = int(limit_text)
    if tool_limit <= 0:
        raise ValueError("TOOL_LIMIT 必须是正整数")
    print("配置文件:", config_path)
    print("助手名称:", agent_name)
    print("状态地址（仅展示，不发请求）:", status_url)
    print("原始 TOOL_LIMIT:", repr(limit_text), type(limit_text).__name__)
```

```
10  def main() -> None:
30      # 配置解析得到字符串，不会自动把 '3' 变成整数，更不会校验业务上的正数范围。
31      tool_limit = int(limit_text)
32      if tool_limit <= 0:
33          raise ValueError("TOOL_LIMIT 必须是正整数")
34      print("配置文件:", config_path)
35      print("助手名称:", agent_name)
36      print("状态地址（仅展示，不发请求）:", status_url)
37      print("原始 TOOL_LIMIT:", repr(limit_text), type(limit_text).__name__)
38      print("转换后的工具数量上限:", tool_limit, type(tool_limit).__name__)
39      print("本例只验证依赖安装与配置读取，不执行 HTTP、LLM 或并发工具调用。")
40
41
42  if __name__ == "__main__":
43      main()
44
```


资源管理器
python-quickstart-f...
java_to_python
chapters
08_oop
README.md
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
demo_project
.python-version
main.py
pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
12_http_json
13_typing
14_data_models
15_protocols_generics
example.py
大纲
时间线
Java Projects
example.py .../14_data_models
example.py .../15_protocols_generics
example.py .../16_decorators
pyproject.toml
java_to_python > chapters > 11_environments_packages > demo_project > pyproject.toml
1 [project]
2 name = "chapter11-config-demo"
3 version = "0.1.0"
4 description = "Read a local Agent configuration without making network requests"
5 requires-python = ">=3.14"
6 dependencies = ["python-dotenv>=1.2.3,<2"]
7
8 [dependency-groups]
9 # 本章没有必须新增的开发工具；以后测试依赖放这里，不混入运行依赖。
10 dev = []
11
12 [tool.uv]
13 # 这是直接运行 main.py 的独立应用，不需要把应用自身构建安装为发行包。
14 package = false
15


python-quickstart-for-java
资源管理器
python-quickstart-f...
java_to_python
chapters
08_oop
README.md
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
demo_project
.python-version
main.py
pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
12_http_json
13_typing
14_data_models
15_protocols_generics
example.py
大纲
时间线
Java Projects
example.py .../14_data_models
example.py .../15_protocols_generics
example.py .../16_decorators
requirem
java_to_python > chapters > 11_environments_packages > demo_project > requirements.txt
1 # 仅用于传统 pip 实验：这是本例的直接运行依赖清单，不是 pip freeze 的输出。
2 # uv 实验只以 pyproject.toml 和 uv.lock 为依据。
3 python-dotenv==1.2.3
4


python-quickstart-for-java
资源管理器
python-quickstart-f...
java_to_python
chapters
08_oop
README.md
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
demo_project
.python-version
main.py
pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
12_http_json
13_typing
14_data_models
15_protocols_generics
example.py
大纲
时间线
Java Projects
example.py .../14_data_models
example.py .../15_protocols_generics
example.py .../16_decorators
java_to_python > chapters > 11_environments_packages > demo_project > settings.env.example
1 # 可提交的演示配置，不包含真实 API Key，也不会访问下面的地址。
2 AGENT_NAME="文档与记录查询助手"
3 STATUS_URL=http://127.0.0.1:8000/status
4 TOOL_LIMIT=3
5

```
version = 1
revision = 3
requires-python = ">=3.14"

[[package]]
name = "chapter11-config-demo"
version = "0.1.0"
source = { virtual = "." }
dependencies = [
    { name = "python-dotenv" },
]

[package.metadata]
requires-dist = [{ name = "python-dotenv", specifier = ">=1.2.3,<2" }]

[package.metadata.requires-dev]
dev = []

[[package]]
name = "python-dotenv"
version = "1.2.3"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/6a/53/ed9d74092561d4b01a2ef1349d52cdbc135e526c245f366b089cf
wheels = [
    { url = "https://files.pythonhosted.org/packages/0d/17/c5c6b53ddc18f297992099b3d9ec16c855c0ccc83263a21fe4d1c625e
]

```


python-quickstart-for-java
资源管理器
python-quickstart-f...
java_to_python
chapters
08_oop
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
demo_project
.python-version
main.py
pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
12_http_json
13_typing
14_data_models
15_protocols_generics
example.py
大纲
时间线
Java Projects
example.py .../14_data_models
example.py .../15_protocols_generics
example.py .../16_decorators
java_to_python > chapters > 11_environments_packages > .gitignore
1 .venv/
2 __pycache__/
3 *.py[cod]
4 .uv-cache/
5 .ruff_cache/
6 .pytest_cache/
7 .env
8 .env.*
9 !.env.example
10
~/vscode/python-quickstart-for-java/java_to_python/chapters/11_environments_packages/demo_project/requirements.txt

```python
"""第 11 章：只读诊断当前解释器、安装位置和仓库声明，不安装任何软件。"""

# 本文件按“解释器 -> 项目声明 -> 安装目录与导入路径 -> 包与模块 -> pip”的顺序收集证据。
# 所有检查都围绕当前 Python 进程展开：同一份脚本换一个解释器启动，看到的环境可能完全不同。
# 这里不会创建环境或安装依赖；唯一启动的子进程也只是执行一次只读的 pip 版本查询。

# 下列模块全来自标准库，不需要 pip 安装：
# importlib.util 查询导入名可能从哪里加载；os 读取进程环境变量；shutil 按 PATH 查命令；
# site 与 sysconfig 说明用户包开关和默认安装路径；subprocess 启动并观察子进程；
# sys 提供当前解释器、环境前缀、版本和模块搜索路径；tomllib 读取 TOML；
# importlib.metadata 读取已安装发行包的元数据；Path 以对象方式拼接和检查文件系统路径。
import importlib.util
import os
import shutil
import site
import subprocess
import sys
import sysconfig
import tomllib
from importlib import metadata
from pathlib import Path

# __file__ 是当前脚本文件的路径。resolve() 转成规范的绝对路径，parent 再取得本章目录；
# 因而后面读取仓库文件不依赖“从哪个工作目录启动脚本”。全大写只表示约定上的模块常量，
# Python 并不会禁止后续代码重新绑定它们。
CHAPTER_DIRECTORY: Path = Path(__file__).resolve().parent
# parents[0] 是 chapters，parents[1] 是 java_to_python，parents[2] 才是仓库根目录。
REPOSITORY_DIRECTORY: Path = CHAPTER_DIRECTORY.parents[2]


# title: str 与 -> None 是类型提示：约定调用者传入字符串，函数只产生输出、不返回业务值；
# Python 运行时默认不会像 Java 编译器那样仅凭这些标注强制检查类型。
def print_section(title: str) -> None:
    """让终端输出可以逐段对应 README 中的诊断步骤。"""
    # f-string 会计算花括号里的 title 并嵌入字符串；开头的 \n 先留一个空行，分隔相邻章节。
    print(f"\n=== {title} ===")

```

```python


# title: str 与 -> None 是类型提示：约定调用者传入字符串，函数只产生输出、不返回业务值；
# Python 运行时默认不会像 Java 编译器那样仅凭这些标注强制检查类型。
def print_section(title: str) -> None:
    """让终端输出可以逐段对应 README 中的诊断步骤。"""
    # f-string 会计算花括号里的 title 并嵌入字符串；开头的 \n 先留一个空行，分隔相邻章节。
    print(f"\n=== {title} ===")


# 这一段先确认“谁在执行”，因为后续看到的包、路径和 pip 都由这个解释器决定。
def show_interpreter() -> None:
    """用当前进程的信息判断环境，而不是猜测终端提示符。"""
    print_section("1. 谁在运行这份代码")
    # 不对 sys.executable 做 resolve()：环境解释器可能是指向基础解释器的符号链接。
    # 把链接解析到底，反而会丢失本次启动用的是哪一个环境路径的线索。
    print("sys.executable:", sys.executable)
    # sys.version 还包含构建信息；split() 按空白拆成列表，[0] 只取开头的版本号。
    print("Python 版本:", sys.version.split()[0])
    # 虚拟环境中，prefix 通常指向当前环境，base_prefix 指向创建它的基础 Python。
    # 二者不相等是比命令行提示符或 VIRTUAL_ENV 更直接的“当前进程在虚拟环境中”证据。
    print("sys.prefix:", sys.prefix)
    print("sys.base_prefix:", sys.base_prefix)
    print("是否处于虚拟环境:", sys.prefix != sys.base_prefix)
    # os.environ 像只含字符串键值的映射；get() 在键不存在时返回这里给定的说明文字，
    # 不会像 os.environ["VIRTUAL_ENV"] 那样抛 KeyError。直接调用 .venv/bin/python 时，
    # 解释器仍会识别虚拟环境，但这个主要由激活脚本设置的变量可以不存在。
    print("VIRTUAL_ENV:", os.environ.get("VIRTUAL_ENV", "未设置；直接调用环境解释器也能运行"))
    # cwd() 是进程的当前工作目录；CHAPTER_DIRECTORY 是源码所在目录，二者并非同一个概念。
    print("当前工作目录:", Path.cwd())
    print("脚本所在目录:", CHAPTER_DIRECTORY)
    # version_info 类似一个可按位置读取的版本记录；[:2] 取 major、minor 组成的新序列，
    # 再与二元组 (3, 14) 比较，因此 3.14 的任意补丁版本都会得到 True。
    print("是否为本章规划的 Python 3.14:", sys.version_info[:2] == (3, 14))


```

```python
64
65  # 这一段把“项目声明的要求”和“机器上实际存在、当前正在使用的环境”并排展示。
66  # 声明文件只是可重建环境的输入；读取到 requires-python 或 dependencies 不等于已经安装。
67  def show_repository() -> None:
68      """根据脚本位置读取仓库资料，所以切换工作目录不会读错项目。"""
69      print_section("2. 仓库要求与已有 .venv")
70      # Path 的 / 运算符是路径拼接，不是字符串除法；结果仍是一个 Path 对象。
71      project_path: Path = REPOSITORY_DIRECTORY / "pyproject.toml"
72      # with 是上下文管理器语法，代码块结束时会自动关闭文件，作用类似 Java 的
73      # try-with-resources。tomllib.load() 要求二进制文件对象，所以这里使用 "rb"。
74      with project_path.open("rb") as file:
75          # TOML 的表会转成嵌套字典，例如 [project] 对应 configuration["project"]。
76          configuration: dict[str, Any] = tomllib.load(file)
77      # 方括号取必需项，缺失时抛 KeyError，能暴露仓库结构不符合本章预期；
78      # get() 则用于可选项，并分别以空列表、空字典表达“当前没有声明”。
79      project: Any = configuration["project"]
80      print("项目文件:", project_path)
81      print("项目名:", project["name"])
82      print("requires-python:", project["requires-python"])
83      print("直接运行依赖:", project.get("dependencies", []))
84      print("开发依赖组:", configuration.get("dependency-groups", {}))
85
86      # is_dir() 只回答仓库根目录下是否真的存在 .venv 目录，不会创建它。
87      environment_path: Path = REPOSITORY_DIRECTORY / ".venv"
88      print("仓库 .venv 存在:", environment_path.is_dir())
89      # 两边都 resolve() 后再按路径值比较，避免相对路径或符号链接形式不同造成误判。
90      # 这里比较的是当前进程的环境前缀，并不是根据 VIRTUAL_ENV 是否设置来猜测。
91      print("当前进程使用仓库 .venv:", Path(sys.prefix).resolve() == environment_path.resolve())
92      environment_config: Path = environment_path / "pyvenv.cfg"
93      # pyvenv.cfg 是创建虚拟环境时生成的文本配置；只有文件真实存在才读取。
94      if environment_config.is_file():
95          # read_text() 一次读入字符串，splitlines() 再按行组成列表供 for 循环逐项处理。
96          # 只显示与解释器隔离相关的字段，不执行文件中的任何内容。
97          for line in environment_config.read_text(encoding="utf-8").splitlines():
98              # partition("=") 总返回三部分，并按位置解包给三个名字；若没有等号，
99              # separator 会是空字符串。这里只在分隔符存在且键属于白名单时输出。
100             key: str, separator: str, value: str = line.partition("=")
```

```python
def show_repository() -> None:
    environment_config: Path = environment_path / "pyvenv.cfg
    # pyvenv.cfg 是创建虚拟环境时生成的文本配置；只有文件真实存在才读取。
    if environment_config.is_file():
        # read_text() 一次读入字符串，splitlines() 再按行组成列表供 for 循环逐项处理。
        # 只显示与解释器隔离相关的字段，不执行文件中的任何内容。
        for line in environment_config.read_text(encoding="utf-8").splitlines():
            # partition("=") 总返回三部分，并按位置解包给三个名字；若没有等号，
            # separator 会是空字符串。这里只在分隔符存在且键属于白名单时输出。
            key: str, separator: str, value: str = line.partition("=")
            if separator and key.strip() in {"home", "version", "include-system-site-packages"}:
                # strip() 去掉等号两侧可能存在的空白，但不会改动磁盘上的配置文件。
                print(f"  {key.strip()}: {value.strip()}")
    else:
        # else 与上面的 if 配对：缺少配置时给出诊断信息，而不是擅自修复环境。
        print("没有找到 pyvenv.cfg；本脚本只报告，不创建或修复环境。")


# 安装工具把文件放在哪里，与 import 最终按什么顺序查找，是两个相关但不同的问题。
def show_installation_paths() -> None:
    """区分默认安装目录和实际导入搜索路径。"""
    print_section("3. 依赖通常装在哪里，import 又到哪里找")
    # purelib 通常放纯 Python 包，platlib 面向平台相关扩展；在当前平台二者也可能相同。
    # get_path() 给出当前解释器安装方案的默认位置，不保证每个模块实际都来自那里。
    print("纯 Python 包默认安装位置 purelib:", sysconfig.get_path("purelib"))
    print("平台相关扩展默认安装位置 platlib:", sysconfig.get_path("platlib"))
    # ENABLE_USER_SITE 表示当前解释器是否会启用用户级 site-packages，不表示其中一定有包。
    print("用户 site-packages 是否启用:", site.ENABLE_USER_SITE)
    print("sys.path（仅观察，不修改）:")
    # sys.path 是 import 的模块搜索路径列表，前面的条目通常比后面的优先。
    # enumerate() 每轮同时给出从 0 开始的位置和路径，不必手工维护计数器。
    for position, entry in enumerate(sys.path):
        # “entry or 说明文字”利用空字符串为假值的规则，仅把空条目换成易读说明；
        # 非空路径保持原值。这里没有修改 entry，更没有修改 sys.path。
        print(f"  [{position}] {entry or '(空字符串表示当前工作目录)'}")
    print("上述安装位置是默认方案；实际模块来源仍须检查 origin 或 __file__。")
```

```python

# 发行包名属于安装工具和元数据系统，模块名属于 import 系统；二者没有机械的一一映射。
# 本例先按发行包名查安装记录，再按可能的导入名查位置，让两类证据可以直接对照。
def show_distribution_and_import_names() -> None:
    """分别检查安装元数据与模块位置，避免把安装名直接当成导入名。"""
    print_section("4. 发行包元数据与可导入模块")
    # 圆括号中的字符串构成 tuple；for 每轮把其中一个发行包名绑定到 distribution_name。
    for distribution_name in ("pip", "python-dotenv", "ruff", "pyright"):
        try:
            # distribution() 读取当前解释器可见的安装元数据，不会联网查询包索引。
            distribution: Distribution = metadata.distribution(distribution_name)
        except metadata.PackageNotFoundError:
            # 只捕获“没有该发行包元数据”这一种预期情况；其他异常仍会暴露出来。
            print(f"发行包 {distribution_name}: 当前解释器未发现安装元数据")
        else:
            # try 没有抛出上面的异常才进入 else，此时 distribution 一定已经成功绑定。
            # version 来自安装元数据；locate_file("") 把空的相对路径定位到分发根目录。
            print(f"发行包 {distribution_name}: {distribution.version}")
            print("  元数据所对应的安装根目录:", distribution.locate_file(""))

    # 只查顶层模块的位置，不 import 第三方包，不触发它们的顶层业务代码。
    # find_spec 对带点的名字可能先导入父包，本例故意只使用不带点的导入名。
    for module_name in ("dotenv", "python_dotenv"):
        # 找到时返回 ModuleSpec（模块加载说明），找不到时返回 None；这一步不会帮忙安装包。
        specification: ModuleSpec | None = importlib.util.find_spec(module_name)
        # 条件表达式按“满足条件时的值 if 条件 else 否则的值”阅读。
        # 先判断 None 很重要，否则对 None 读取 .origin 会抛 AttributeError。
        origin: str | None = specification.origin if specification is not None else "未找到"
        print(f"导入名 {module_name}: {origin}")
    print("元数据存在和位置可发现都不等于实际 import 已成功；完整运行见独立实验。")


# 这一段同时观察 PATH 中的独立命令，以及“当前解释器 -m pip”的绑定关系。
# 前者可能指向别的环境；后者明确要求 sys.executable 这一个 Python 运行 pip 模块。
def show_pip_binding() -> None:
    """用同一个解释器启动 pip 的只读版本查询，且为子进程设置时间上限。"""
    print_section("5. python -m pip 与解释器的对应关系")
```

```python
# 这一段同时观察 PATH 中的独立命令，以及“当前解释器 -m pip”的绑定关系。
# 前者可能指向别的环境；后者明确要求 sys.executable 这一个 Python 运行 pip 模块。
def show_pip_binding() -> None:
    """用同一个解释器启动 pip 的只读版本查询，且为子进程设置时间上限。"""
    print_section("5. python -m pip 与解释器的对应关系")
    # which() 只按传给当前进程的 PATH 查找可执行文件，返回路径或 None；它看不到
    # shell 自己维护的 alias/function，因此终端中还要用 type 命令补充判断。
    print("当前 shell PATH 中的 python:", shutil.which("python"))
    print("当前 shell PATH 中的 pip:", shutil.which("pip"))
    print("当前 shell PATH 中的 uv:", shutil.which("uv"))
    print("以上是 PATH 查找结果；shell 的别名或函数还需在终端用 type 检查。")
    # 参数用 list 分开保存，不经过 shell 字符串解析：第一项锁定当前解释器；-B 禁止写
    # .pyc；-m pip 让该解释器按模块运行 pip；--version 只查询版本，不执行安装。
    command: list[str] = [sys.executable, "-B", "-m", "pip", "--version"]
    print("执行的参数列表:", command)
    try:
        # capture_output=True 分别收集 stdout/stderr；text=True 把结果解码为 str；
        # timeout=10 限制等待时间；check=False 让非零退出码保存在结果中而不是抛异常。
        result: CompletedProcess[str] = subprocess.run(command, capture_output=True, text=True, timeout=10, check=Fa
    except subprocess.TimeoutExpired:
        # 超时是 run() 会抛出的特定异常。打印原因后 return，只结束当前函数；
        # main() 随后仍会继续输出第 6 段总结。
        print("pip 版本查询超过 10 秒，已终止子进程；请检查本地环境。")
        return
    # returncode 为 0 通常表示命令成功。stdout or stderr 优先使用非空的标准输出，
    # 否则展示标准错误；strip() 仅移除展示文本首尾的空白与换行。
    print("退出码:", result.returncode)
    print((result.stdout or result.stderr).strip())
    # 因为 check=False，普通失败不会自动成为异常，要显式检查退出码并解释常见情况。
    if result.returncode != 0:
        print("uv 创建的环境可以没有 pip；应结合错误正文判断，不要立即重装 Python。")


# main() 是流程编排层：前五个函数按顺序完成只读采集，最后给出排查优先级。
def main() -> None:
    """默认入口只收集环境信息；安装实验必须由读者显式执行 README 的命令。"""
```

```python
def show_pip_binding() -> None:
    print("退出码:", result.returncode)
    print((result.stdout or result.stderr).strip())
    # 因为 check=False，普通失败不会自动成为异常，要显式检查退出码并解释常见情况。
    if result.returncode != 0:
        print("uv 创建的环境可以没有 pip；应结合错误正文判断，不要立即重装 Python。")


# main() 是流程编排层：前五个函数按顺序完成只读采集，最后给出排查优先级。
def main() -> None:
    """默认入口只收集环境信息；安装实验必须由读者显式执行 README 的命令。"""
    # Python 普通函数调用是顺序执行的；上一段完成（或正常 return）后才进入下一段。
    show_interpreter()
    show_repository()
    show_installation_paths()
    show_distribution_and_import_names()
    show_pip_binding()
    print_section("6. 下一步如何判断")
    print("先比较终端与 IDE 的 sys.executable，再比较安装元数据、模块来源和工作目录。")
    print("本次没有创建环境、修改声明、联网安装或执行第三方业务模块。")


# Python 执行脚本时把 __name__ 设为 "__main__"，所以会调用 main()；若别的模块只做
# import example，__name__ 是模块名，这个入口不会自动运行，函数定义仍可被单独复用。
if __name__ == "__main__":
    main()
```

```python
"""显式运行的离线排错实验：当前环境须已安装 python-dotenv，临时资源自动清理。"""

import subprocess
import sys
import tempfile
from pathlib import Path

GOOD_IMPORT = """
import sys
import dotenv
from importlib.metadata import version
print("解释器:", sys.executable)
print("发行包版本:", version("python-dotenv"))
print("模块来源:", dotenv.__file__)
"""


def run_probe(
    interpreter: str,
    code: str,
    directory: Path,
    *,
    expected_error: str | None = None,
    isolated: bool = True,
) -> None:
    """每次使用新进程，让导入缓存与反例文件不会影响下一次检查。"""
    # -I 排除当前目录、用户包目录及 PYTHON* 环境变量对导入的干扰。
    # 名称遮蔽实验改用 -E -s，保留 -c 启动时的当前目录搜索入口。
    flags: list[str] = ["-I"] if isolated else ["-E", "-s"]
    result: CompletedProcess[str] = subprocess.run(
        [interpreter, "-B", *flags, "-c", code],
        cwd=directory,
        capture_output=True,
        text=True,
        timeout=10,
        check=False,
    )
```

```python
def run_probe(
    )
    print(result.stdout.strip())
    print("退出码:", result.returncode)
    if expected_error is None:
        if result.returncode != 0:
            raise RuntimeError(f"预期导入成功，却失败：\n{result.stderr}")
    else:
        if result.returncode == 0 or expected_error not in result.stderr:
            raise RuntimeError(f"未出现预期的 {expected_error!r}：\n{result.stderr}")
        print("已观察到预期错误:", result.stderr.strip().splitlines()[-1])


def main() -> None:
    with tempfile.TemporaryDirectory(prefix="chapter11-import-") as directory:
        root = Path(directory)
        print("\n=== 1. 环境 A 已安装，实际导入成功 ===")
        run_probe(sys.executable, GOOD_IMPORT, root)

        print("\n=== 2. 反例：把发行包名机械地改成下划线作为导入名 ===")
        run_probe(
            sys.executable,
            "import python_dotenv",
            root,
            expected_error="No module named 'python_dotenv'",
        )

        print("\n=== 3. 反例：安装在 A，却用空环境 B 运行 ===")
        empty_environment: Path = root / "empty-environment"
        # B 只负责证明环境隔离，不负责安装，所以不需要 pip；创建过程不联网。
        subprocess.run(
            [sys.executable, "-B", "-m", "venv", "--without-pip", str(empty_environment)],
            check=True,
            capture_output=True,
            text=True,
            timeout=30,
        )
```

```
49  def main() -> None:
72          )
73          executable_parts: tuple[Literal['bin'], Literal['python']] = (
74              ("Scripts", "python.exe") if sys.platform == "win32" else ("bin", "python")
75          )
76          empty_python: Path = empty_environment.joinpath(*executable_parts)
77          run_probe(
78              str(empty_python),
79              "import sys; print('解释器:', sys.executable); import dotenv",
80              root,
81              expected_error="No module named 'dotenv'",
82          )
83
84          print("\n=== 4. 反例：同一环境里，本地同名文件遮蔽了正确的包 ===")
85          shadow_directory: Path = root / "shadow"
86          shadow_directory.mkdir()
87          (shadow_directory / "dotenv.py").write_text(
88              'SOURCE = "这是反例文件，不是已安装的 dotenv 包"\n', encoding="utf-8"
89          )
90          run_probe(
91              sys.executable,
92              "import dotenv; print('实际来源:', dotenv.__file__); from dotenv import dotenv_values",
93              shadow_directory,
94              expected_error="cannot import name 'dotenv_values'",
95              isolated=False,
96          )
97
98          print("\n=== 5. 回到无同名文件的目录，环境 A 再次导入成功 ===")
99          run_probe(sys.executable, GOOD_IMPORT, root)
100
101     print("临时空环境与反例文件已清理:", not root.exists())
102
103
104 if __name__ == "__main__":
105     main()
106
```